HMR3D Current Version Summary
Implementation and Experimental Results

Repository snapshot: commit `46f5e73`

April 11, 2026

1 Scope

This document summarizes the currently committed HMR3D implementation in `/home/pengyue/Codespace/HMR3D`. It intentionally covers only the committed `pose-anchor` version and excludes later local `Gaussian/submap` experiments that were not kept in the clean working tree.

The current system extends the upstream `TTT3R` runtime with an explicit state-level memory lifecycle:

- archive recurrent state snapshots into an external bank,
- retrieve candidate memories by descriptor similarity,
- verify candidates with a counterfactual geometric rollout, and
- further filter ambiguous or stale recoveries with anchor-pose scoring.

The best validated configuration is the experiment manifest `configs/tum_relpose_sweep_224_v11_pose_anch` evaluated against the `TTT3R` baseline on seven TUM long-sequence windows under deterministic inference.

2 Implementation Summary

2.1 Code Organization

The current implementation is split across the following local modules:

- `hmr3d_memory/config.py`: memory hyper-parameters and mode-specific configuration.
- `hmr3d_memory/router.py`: archive bank, retrieval logic, recovery proposal construction, geometric verification statistics, and anchor-pose verification statistics.
- `hmr3d_memory/adapter.py`: runtime wrapper around the upstream `TTT3R` rollout, including the counterfactual verification path.
- `hmr3d_memory/eval_relpose.py`: deterministic rel-pose evaluation, per-sequence metric aggregation, and memory event dumps.
- `scripts/run_relpose_memory_sweep.py`: experiment sweep entrypoint.
- `scripts/run_longseq_eval.py`: full long-sequence evaluation entrypoint.

2.2 Memory Lifecycle

At the current commit, the memory system is state-level and does not modify the upstream submodule parameters.

Archive. Every active segment maintains frame descriptors and the current recurrent state. When the archive interval is reached, the system stores:

- a segment descriptor,
- a short tail descriptor history,
- a combined state descriptor from `state_feat` and `mem`,
- the recurrent tensors needed to reconstruct the state, and
- an optional camera pose anchor.

This is implemented in `MemoryRouter` via `ArchiveEntry`.

Retrieve. At each eligible frame, the router ranks archived segments with descriptor similarity. It then applies thresholding on:

- query-to-archive descriptor similarity,
- current-state similarity,
- the query-state gap, and
- a recovery cooldown and an attempt cooldown.

Geometric verification. For shortlisted candidates, the adapter runs a counterfactual rollout: it injects the archived state proposal, executes the current frame, and compares the resulting geometric quality against the baseline rollout. The candidate is accepted only if it satisfies the configured geometric and confidence tests.

Anchor-pose verification. The current best version adds a second verifier for ambiguous or stale candidates. It compares the current pose estimate and the archived camera anchor through an anchor-pose quality score. The recovery is retained only if the candidate score is at least as good as the baseline score under the configured ratio and gain thresholds.

2.3 Current Default Hyper-parameters

The best validated manifest uses the following memory settings:

Parameter	Value
`archive_interval`	120
`retrieval_topk`	2
`retrieval_similarity_thresh`	0.92
`verification_similarity_thresh`	0.80
`max_state_similarity_for_recover`	0.995
`query_state_gap_thresh`	-0.05
`recovery_alpha`	0.35
`verification_geo_ratio_thresh`	1.0
`verification_conf_tolerance`	0.1
`min_frames_before_retrieve`	30
`retrieval_cooldown`	30
`retrieval_attempt_cooldown`	30
`reset_on_archive`	false
`enable_anchor_pose_verification`	true
`anchor_pose_only_for_ambiguous`	true
`anchor_pose_rotation_weight`	0.25
`anchor_pose_score_ratio_thresh`	1.0
`anchor_pose_min_score_gain`	0.0

3 Evaluation Protocol

3.1 Dataset and Hardware

The current report uses the deterministic rel-pose sweep at:

- dataset: `tum_s1_1000`,
- input size: `224`,
- device: `cuda`,
- seed: `0`,
- deterministic: `true`.

The seven evaluated sequence windows are:

- `sitting_static`,
- `static_head500`,
- `static_tail500`,
- `sitting_xyz`,
- `xyz_head500`,
- `xyz_mid500`,
- `xyz_tail500`.

The evaluation files are stored under `reports/generated/tum_relpose_sweep_224_v11_pose_anchor_full7`.

3.2 Compared Methods

Three configurations were compared:

- **TTT3R baseline**: upstream behavior without external archive/retrieve/recover.
- **HMR geometry-only**: descriptor retrieval plus geometric verification.
- **HMR pose-anchor**: geometry-only plus anchor-pose verification on ambiguous or stale candidates.

4 Main Results

4.1 Aggregate Results

Table 1 reports the main aggregate metrics.

Table 1: Aggregate results on the seven-window TUM rel-pose benchmark. Lower is better for ATE, RPE_trans, and RPE_rot. Higher is better for FPS.

Method	Avg. ATE	Avg. $\mathrm{RPE}_{\mathrm{trans}}$	Avg. $\mathrm{RPE}_{\mathrm{rot}}$	Avg. FPS
TTT3R baseline	0.088360	0.025146	0.938209	3.779
HMR geometry-only	0.081347	0.025555	0.938447	3.716
HMR pose-anchor	**0.079694**	0.025560	**0.934255**	3.668

Relative to the baseline, the current **HMR pose-anchor** version:

- reduces average ATE from 0.088360 to 0.079694,
- yields a relative ATE improvement of approximately 9.8%,
- slightly improves average rotational error, and
- incurs an average throughput drop of about 2.9%.

Relative to the geometry-only version, pose-anchor further improves ATE by approximately 2.0%.

4.2 Per-sequence Results

Table 2 shows the per-window ATE comparison between the baseline and the current best HMR version.

The key pattern is stable:

- 6 of 7 windows improve in ATE,
- the largest gain appears on `xyz_tail500`,
- the only regression is `xyz_mid500`, and
- the method is improving global trajectory quality more clearly than local translational error.

4.3 Memory Activity

The best HMR trial generated the following aggregate memory statistics:

- 33 archives,
- 108 retrieval attempts,
- 32 successful recoveries after verification,
- 124 geometric verification rollouts,
- 32 anchor-pose verification accepts,
- 92 anchor-pose verification rejects,
- average recovery success rate of 29.6%.

These numbers show that the gain does not come from blindly restoring history. The majority of retrieval candidates are rejected by the verification layers, and only a filtered subset is merged back into the active state.

5 Interpretation

The current version supports the following conclusions.

What works. The archive/retrieve/recover lifecycle can improve long-sequence robustness without retraining the base model. The main benefit is on **global ATE**, which is consistent with the goal of mitigating drift and exploiting long-range revisits.

What does not improve yet. Average translational RPE does not improve. This means the method is not yet a uniform improvement across all metrics. It is currently stronger as a **global drift correction mechanism** than as a local-frame refinement mechanism.

Why pose-anchor helps. The geometry-only variant already removes many bad recoveries, but ambiguous revisits still remain. Adding anchor-pose verification narrows those cases and reduces average ATE from 0.081347 to 0.079694 on the same full-seven benchmark.

6 Reproducibility

The exact experiment manifest for the current best result is:

```
configs/tum_relpose_sweep_224_v11_pose_anchor_full7.json
```

The sweep can be reproduced with:

```
python3 scripts/run_relpose_memory_sweep.py -config
configs/tum_relpose_sweep_224_v11_pose_anchor_full7.json
```

The key output files are:

- `reports/generated/tum_relpose_sweep_224_v11_pose_anchor_full7/leaderboard.json`
- `reports/generated/tum_relpose_sweep_224_v11_pose_anchor_full7/ttt3r_baseline/summary.json`
- `reports/generated/tum_relpose_sweep_224_v11_pose_anchor_full7/hmr_full_geometry_pose_ancho`

7 Current Limitations

This committed version still has clear boundaries:

- it is a state-level memory system, not yet a map-level submap system,
- it improves ATE more consistently than $\mathrm{RPE}_{\mathrm{trans}}$,
- one of the seven windows still regresses, and
- the evaluation was run at input size 224 because the local RTX 2060 6GB machine could not reliably run the same long-sequence setup at 512.

8 Bottom Line

For the current clean repository state, the correct short summary is:

> The committed HMR3D implementation adds an external state-memory lifecycle to TTT3R and achieves a **9.8% average ATE improvement** over the TTT3R baseline on a deterministic seven-window TUM long-sequence benchmark, with a small throughput cost and no corresponding gain in translational RPE.

Table 2: Per-sequence ATE comparison. Positive "Rel. gain" means the HMR version improves over the baseline.

Sequence	Baseline ATE	HMR ATE	ΔATE	Rel. gain (%)
`sitting_static`	0.028312	0.021613	-0.006700	23.66
`static_head500`	0.023047	0.021860	-0.001187	5.15
`static_tail500`	0.014938	0.013395	-0.001544	10.33
`sitting_xyz`	0.278476	0.269411	-0.009065	3.26
`xyz_head500`	0.064023	0.054727	-0.009297	14.52
`xyz_mid500`	0.067648	0.070329	0.002681	-3.96
`xyz_tail500`	0.142074	0.106521	-0.035553	25.02